A) Executive Summary

Product

SokMed is a multi-tenant web app for managing social media content creation and approvals as **single "Post Tickets."** Each post ticket becomes the **single source of truth**: design asset(s), caption, metadata, one comment thread, workflow status, and a complete audit trail.

Core Workflows

1. **Plan & create posts** (manually or via CSV import) → posts appear on the **Kanban board** in a default status (e.g., Planning).
2. **Execute content**: contributors upload/replace design versions and edit captions with versioning.
3. **Request approval**: internal supervisors and external clients receive **approval emails** with:
    - design preview
    - caption text
    - **Approve** (one-click, token-based)
    - **Comment** (deep link to post page, focused comment box)
4. **Revise loop**: feedback is centralized in one thread, status moves are drag-and-drop and auditable.
5. **Track accountability**: every action is logged (status change, edits, notifications, approvals).
6. **Export/Reporting**: export CSV calendar filtered by company/brand/date range, plus analytics events to measure success.

Success Metrics (Tracked)

- **Median time-to-approval** (approval cycle time)
- **% approvals via email** (one-click vs. needing follow-up)
- **"External follow-up" events per post** (proxy for scattered comms)
- **Audit completeness** (100% log coverage for core actions)

B) Glossary + Definitions

Term	Definition
Organization / Tenant	A top-level isolated entity in SokMed (multi-tenant). All data is scoped by organization_id.
Company	A sub-entity under Organization (MVP uses **Option A**: Organization == Company).
Brand	A marketing brand within an Organization/Company; posts belong to a brand.
Post	A single planned social media item with metadata, design assets, caption versions, approvals, and status.
Ticket	Synonym for Post in MVP: **one post = one ticket**.
Status	A workflow state shown as a Kanban column (customizable per organization).
Kanban Board	Board view grouping posts by status; supports drag-and-drop movement.
Approver (Supervisor)	Internal user with approval responsibility (Manager/Supervisor role).
Client Approver	External party who can approve/comment on assigned posts (email token + optional login).
Approval Request	An action triggering an approval email to one or more approvers with secure links.
Approval Decision	The recorded outcome from an approver: **Approved** or **Request Changes** (MVP requires at least Approved; Request Changes is recommended).
Audit Event	Immutable record of any relevant action (status move, upload, edit, comment, notify, approval).
Caption Version	A stored snapshot of caption text each time it changes.

Asset Version	A stored snapshot of design/banner file metadata and storage path each time design is uploaded/replaced.
SLA/Deadline	A required-by timestamp for design completion and caption completion per post.
Post Date/Time	Intended publish/scheduled datetime of the post; can be **TBD** for MVP.
Tenant Isolation	Ensuring users of one organization cannot access data of another (via RLS + scoped API).
RLS	Supabase Row Level Security policies enforcing tenant and role-based permissions at DB layer.

C) Stakeholder-Friendly User Stories

Organization Owner (Admin)

- As an Owner, I want to configure **statuses/columns** so our workflow matches our process.
- As an Owner, I want to manage brands and users so only the right people see the right posts.
- As an Owner, I want audit logs so I can resolve disputes and measure accountability.

Manager/Supervisor (Approver)

- As a Supervisor, I want to receive an email with the latest design and caption so I can approve quickly.
- As a Supervisor, I want to comment in-context so revisions are unambiguous.
- As a Supervisor, I want to see who changed what and when to enforce deadlines.

Contributor (Designer/Copywriter/Marketer)

- As a Contributor, I want to upload design versions and see feedback in one thread.

- As a Contributor, I want to edit captions with version history so prior drafts aren't lost.
- As a Contributor, I want clear workflow states so I know what to do next.

Client Approver (External)

- As a Client, I want to approve directly in the email without logging in.
- As a Client, I want to click "Comment" and land on the exact post thread so I can request changes clearly.
- As a Client, I want to only see the posts assigned to my brand(s).

Client Viewer (Out of Scope for MVP)

- As a Viewer, I want read-only access without approvals (phase 2).

D) User Flows (text-based)

1) Kanban management flow (with filtering)

1. User logs in → lands on **Kanban Board**.
2. User selects filters:
    - Company (if multiple; MVP: Organization == Company so filter is present but defaults to org)
    - Brand (dependent on company)
    - Status, Date Range (Post Date), keyword search
3. Board renders columns (statuses) and cards.
4. User drags a card from Status A → Status B.
5. System:
    - validates permission + allowed transition rules (MVP: any to any, configurable later)
    - updates posts.status_id
    - writes post_activity_log event
    - triggers notifications per rules

2) Create post → upload design → write caption → deadlines → notify supervisor → supervisor approves/comments

1. Contributor clicks “New Post” (MVP: quick create modal) OR imports CSV.
2. Sets metadata (brand/theme/channels/goals), sets design and caption deadlines.
3. Uploads initial design asset → system stores in Supabase Storage + creates asset version record.
4. Writes caption → system creates caption version record.
5. Clicks **Notify Supervisor for Approval**
6. System creates approval_request records and sends email(s).
7. Supervisor clicks:
    - **Approve** in email → token verified → approval recorded → status optionally auto-moves to “Approved”
    - or **Comment** → opens post page → comment added → status optionally moves to “Revision”

3) Notify client → client approves via email → audit recorded

1. Contributor/Manager clicks **Notify Client for Approval**
2. System emails client approvers with design+caption+buttons.
3. Client clicks **Approve** → approval recorded without login (token).
4. If policy requires all approvers, post remains “For Review” until all required approvals recorded.

4) Revision loop

1. Approver requests changes (comment or decision).
2. Contributor updates design (new asset version) and/or caption (new version).
3. Contributor re-notifies approvers (throttled unless forced).
4. Audit trail shows each loop.

5) CSV import → posts created → appear on Kanban in Planning

1. User opens Import CSV modal.
2. Uploads CSV → system parses + validates → shows preview with row errors.
3. User confirms → system creates posts for valid rows.
4. Posts appear in default status “Planning”.
5. Import job + row outcomes stored and auditable.

6) CSV export → filtered by company/brand/date range

1. User opens Export CSV modal.
2. Selects filters (company/brand/date range/status).
3. System generates CSV in template format and downloads.

E) UX / UI Specification

Screens List

1. **Login**
2. **Kanban Board** (with filter bar)
3. **Post/Ticket Detail**
4. **Import CSV** (modal/page with preview + validation)
5. **Export CSV** (modal)
6. **Admin: Status Editor**
7. **Admin: Company/Brand Management**
8. **User Management + Client Assignment**

1) Login

Components

- Email + password (Supabase Auth) + SSO optional (out of scope)
- “Forgot password”
- Loading and error states

States

- Invalid credentials → error message
- Unverified email (if enabled) → guidance

2) Kanban Board (with filter bar)

Top Filter Bar

- Company selector (MVP still shows but may be locked if org==company)
- Brand selector (dependent on company)
- Status filter (multi-select optional)
- Date range (Post Date)
- Search input (theme/caption keyword)
- “Import CSV” / “Export CSV”
- “New Post”

Kanban Columns

- Columns from post_statuses ordered by sort_order
- Each column shows count + cards

Card UI (minimum)

- Brand name
- Theme/Title
- Status badge
- Post Date/Time or “TBD”
- Deadline indicators:
    - Design deadline: green/amber/red
    - Caption deadline: green/amber/red

Deadline thresholds (MVP)

- **Amber**: within 24 hours
- **Red**: overdue (current time > deadline)
- If deadline missing: show “No deadline” label

Interactions

- Drag card → drop in new column
- Click card → opens Post Detail
- Empty column state text
- Error state: “Could not load board. Retry.”

3) Post/Ticket Detail

Main area

- Design preview (large)
    - If multiple versions: show “Version dropdown” or “Latest + history list”
- Caption block (readable, copy button)
    - “Edit caption” (contributors/managers)
    - Version history link
- Comment thread (single thread per post)
    - Replies (threaded by parent_id)
    - Mentions optional (phase 2)

Tabs/Sections

- **Discussion**
- **Activity/Audit**

Sidebar (metadata & controls)

- Post Date (or TBD)
- Post Time + timezone
- Deadlines: Design / Content
- Company
- Brand
- Theme
- Framework
- Content type
- Purpose / Primary Goal
- Type of promotion
- Channels (list)
- Target audience
- Supervisor approvers (multi)
- Client approvers (multi)
- Assignees (optional)
- Buttons:
    - Notify Supervisor for Approval
    - Notify Client for Approval

Permission denied

- If user lacks access: show “You don’t have access to this post.”

4) Import CSV Modal/Page

Steps

1. Upload CSV
2. Parse & validate
3. Preview table (first N rows)
4. Errors list by row/column
5. “Confirm import” button
6. Results summary

Preview table columns

- All template columns + status mapping

5) Export CSV Modal

- Filters: Company/Brand/Date range/Status
- “Download CSV”

6) Admin: Status Editor

- List statuses with drag reorder
- Add status, rename, delete
- Validation: cannot delete if posts exist unless re-map target status
- Default statuses seed

7) Admin: Company/Brand Management

- Company list (MVP optional if org==company)
- Brand list CRUD
- Brand metadata (name, color optional)

8) User management + client assignment

- Invite internal users (email invite)
- Assign roles
- Add client contacts (email, name)
- Assign client contacts to brands and/or posts
- Toggle client permissions: approve/comment

Accessibility basics

- Keyboard navigable controls
- ARIA labels for buttons and drag/drop
- Color not sole indicator (deadline icons + text)
- Sufficient contrast for badges and links

F) Functional Requirements (Detailed)

Global Rules (applies to all features)

- **Tenant isolation**: every query scoped by organization_id.
- All critical actions write **audit events**.
- Time handling uses:
  - Organization default timezone (required at org creation)
  - Post-specific timezone optional (MVP: use org timezone only; store on post for future override)
- **Approval mode (MVP choice): "ALL approvers required"**
  - Rationale: reduces ambiguity and prevents accidental partial approvals. Can offer "ANY" as org setting later.

Feature 1: Kanban Board for Content Workflow

Overview

Users view and manage posts in a status workflow. Statuses are customizable per organization. Moving a post creates audit events and may trigger notifications.

Inputs / Outputs

Inputs

- Filters: company, brand, status, date range, keyword
- Drag/drop status change

Outputs

- Filtered board columns and cards
- Status updates and audit logs

- Notifications (optional on movement)

Business Rules

- Status list is org-configurable:
  - post_statuses.is_active = true only
  - ordered by sort_order
- A post always has exactly one current status.
- Drag-and-drop requires permission: Contributor+ can move; Clients cannot.

Validation

- Cannot move to a status that doesn't exist or is inactive.
- If post is locked (optional future), disallow movement.

Permissions

Action	Owner	Manager	Contributor	Client Approver
View board	✅	✅	✅	❌ (clients use deep links / limited view)
Move card	✅	✅	✅	❌
View all brands	✅	✅	✅ (assigned scope)	❌

MVP scope rule for contributors: can see all posts in org (simpler). Phase 2 can restrict by brand/team assignment.

Notifications (movement)

MVP: **No email on drag/move** to reduce noise. Only approval request emails required.
Phase 2: optional "status changed" in-app notifications.

Acceptance Criteria (Given/When/Then)

- **Given** user has access, **when** board loads, **then** show columns in configured order with cards.
- **Given** user drags a card to another column, **when** drop occurs, **then** status updates, audit event created, UI updates without refresh.

- **Given** filters applied, **when** user changes brand filter, **then** board shows only posts in that brand.

Edge Cases

- Empty results: show “No posts match your filters.”
- Status deleted: system must require remap before delete; otherwise block.
- Concurrent updates: if two users move same post, last write wins; UI shows toast “Post updated by another user, refreshed.”

Error States

- Network failure: show retry.
- Permission denied: show “Not allowed.”
- Invalid status ID: show “Status no longer exists.”

Analytics / Audit

- Audit events:
  - post.status_changed with from/to, actor, timestamp
- Analytics events (optional but recommended):
  - kanban.viewed, kanban.filtered, post.dragged

QA Test Cases

- Load board with multiple statuses and posts.
- Drag post across statuses, verify DB update + audit log.
- Apply filters; verify correct subset.
- Permission test: client cannot access board route.

Feature 2: Post/Ticket Detail View (Single Post Page)

Overview

A post is a single ticket containing design preview, caption, discussion, approvals, metadata, and actions to request approvals.

User Flow

1. Click Kanban card → opens post page.
2. View latest design + caption.

3. Comment in thread.
4. Upload/replace design and edit caption (if permitted).
5. Notify supervisor/client for approval.

Inputs / Outputs

Inputs

- Design file upload
- Caption edit
- Comment add
- Notify approval actions
- Post metadata edits (deadlines, date/time, etc.)

Outputs

- Updated latest design/caption
- New versions recorded
- Approval emails sent
- Audit trail entries

Business Rules

- **Single thread per post**; all comments attach to post_id.
- Design assets are versioned; **latest** is post_assets.is_current = true.
- Captions are versioned; **latest** is post_caption_versions.is_current = true.
- Post Date/Time can be:
  - TBD (stored as null datetime + is_tbd=true)
  - scheduled (datetime stored)

Date/Time Rules

- When setting deadlines:
  - Must be >= now (org timezone) unless actor is Owner (retroactive edit allowed).
- If Post Date/Time is set:
  - system warns (soft warning) if Post Date/Time < any deadline.
- Post Time default:
  - if CSV missing time, use org default posting time (e.g., 09:00).

Permissions

Action	Owner	Manager	Contributor	Client Approver
View post (assigned)	✅	✅	✅	✅ (only assigned posts)
Upload/replace design	✅	✅	✅	❌
Edit caption	✅	✅	✅	❌
Comment	✅	✅	✅	✅
Notify supervisor/client	✅	✅	✅	❌
Edit metadata	✅	✅	✅ (limited)	❌
View audit	✅	✅	✅	✅ (limited to visible events)

Client audit visibility (MVP):

- Clients can see:
  - comments
  - approvals involving them
  - status changes (optional; recommend yes)
- Clients cannot see internal-only notes (MVP does not include internal-only comments; phase 2 can add “internal note” type)

Acceptance Criteria

- Opening post shows latest design preview and latest caption.
- Uploading a new design creates a new asset version and updates preview.
- Editing caption creates a new caption version and updates displayed caption.
- Clicking “Notify Supervisor” sends email(s) to supervisor approvers with latest design+caption.
- Clicking “Notify Client” sends email(s) to client approvers with latest design+caption.
- Comment thread shows in chronological order; replies nested.

Edge Cases

- No design uploaded: show placeholder and “Upload design” button (if permitted).
- No caption: show empty state and “Add caption” button.
- Large image: enforce size limits and generate thumbnails.
- Deleted user: audit should keep actor reference as “Deleted user” but preserve ID.

Error States

- Upload fails: show reason (file too large, invalid type).
- Save conflict: show “Newer version exists; refresh to continue.”
- Notify blocked by throttling: show “Recently notified. Try again later or Force send (Manager/Owner only).”

Analytics / Audit

Audit events:

- post.asset_uploaded (asset metadata)
- post.caption_updated (new version id)
- post.comment_added
- post.comment_edited (if allowed)
- post.approval_requested (recipients)
- post.metadata_updated (fields changed)

Analytics:

- post.viewed, post.comment_posted, post.notify_clicked

QA Test Cases

- View post with all sections populated.
- Upload design, verify storage object + DB record + audit.
- Edit caption twice, verify version history.
- Post Date TBD behavior and warnings.
- Client access limited to assigned post.

Feature 3: Activity Tracking / Audit Trail (Per Post)

Overview

Every relevant action becomes an immutable audit event, accessible from the post page.

Rules

- Audit events are append-only.
- Comments can be edited but must preserve original content via:
  - post_comments.edited_at + edit_reason optional
  - store previous content in post_comment_edits (recommended) OR store previous_body field
- Caption changes store full snapshot text per version (diff optional later).
- Asset changes store file metadata snapshot.

Acceptance Criteria

- Activity tab shows chronological list with:
  - event type
  - actor
  - timestamp
  - details (from/to status, recipients, etc.)
- Audit includes email action results: approved via email token, token id, request id.

Edge Cases

- Events generated by system (e.g., auto status move): actor = “System”.
- Email delivery status unknown: log “sent” and optionally provider webhook updates later.

Error States

- Audit retrieval fails: show “Could not load activity.”

QA Test Cases

- Trigger each event type and verify presence and correctness.
- Ensure clients only see allowed audit items.

Feature 4: Notifications

Overview

MVP requires **email notifications for approval requests**. In-app notifications are phase 2.

Event Types and Recipients (MVP)

Event	Recipients	Channel
Supervisor approval requested	All supervisor approvers assigned to post	Email
Client approval requested	All client approvers assigned to post	Email
Comment posted	(MVP) Notify internal users who are assignees + managers + the party being requested	Email optional (phase 2)

MVP decision: Only approval request emails are required. Comment notifications are phase 2 to avoid spam.

Throttling / Anti-spam

- By default, prevent re-sending approval request emails for same post+recipient within **15 minutes**.
- Manager/Owner can “Force send” with reason logged.

Delivery Tracking

- Store:
  - approval_requests.sent_at
  - approval_requests.delivery_status (sent, bounced, deferred) if provider webhooks implemented (optional MVP)

QA Test Cases

- Send supervisor request and confirm email structure + links.
- Throttling prevents rapid re-send.
- Force send works and logs reason.

Feature 5: CSV Import/Export + Auto-fill Kanban + Filtering

A) CSV Import

MVP Approach (Company/Brand mapping)

- CSV must include **Brand** column (required).
- Company column is **optional** in MVP since Organization==Company. If provided, validate equals org/company name or ignore with warning.
- Import UI includes an optional “Default Brand Mapping” step:
    - If brand names don’t match existing brands, user can map CSV values → existing brands or create new brands (Owner/Manager only).

Required Fields (exact list)

- Date
- Brand
- Theme
- Content Type
- Channels
- Target Audience
- Primary Goal
- CTA
- Type of Promotion

Optional:

- Time
- Deadline (Design)
- Deadline (Content)
- Status
- Highlight (80%)
- Cross-Promo (20%)

Mapping Rules

- Date → posts.post_date (date)
- Time → posts.post_time (time); if empty use org default time
- Brand → posts.brand_id (lookup by normalized name)
- Theme → posts.theme

- Content Type → posts.content_type
- Channels → posts.channels (text array; split by delimiter |)
- Target Audience → posts.target_audience
- Primary Goal → posts.primary_goal
- CTA → posts.cta
- Type of Promotion → posts.promotion_type
- Highlight (80%), Cross-Promo (20%) → posts.highlight_text, posts.cross_promo_text
- Status → posts.status_id if matches status name; else default "Planning"

Delimiter choices

- Channels in CSV cell: FB|IG|TikTok
- Export uses same delimiter for round-trip safety.

Import Validation

- Date format accepted:
  - YYYY-MM-DD (preferred)
  - MM/DD/YYYY (accepted; normalized)
- Time accepted:
  - HH:MM 24-hour
  - HH:MM AM/PM
- Deadlines must be parseable datetimes in org timezone, or date+time inferred.

Deduplication / Idempotency

- Dedupe key: organization_id + brand_id + post_date + theme + channels_hash
- Import UI checkbox: "Idempotent import (skip duplicates)" default ON.
- If OFF, duplicates allowed but flagged.

Partial Import

- System imports valid rows and reports failed rows.
- Each row outcome stored in csv_import_job_rows.

Audit

- Job-level audit: who imported, when, counts.
- Row-level audit: created post IDs, errors.

Acceptance Criteria

- Preview shows parsed rows and highlights errors.
- Confirm import creates posts in DB and default status.
- Results screen shows created count, skipped duplicates, failed rows.

QA Test Cases

- Missing required fields → validation error per row.
- Unknown brand → mapping/create flow.
- Duplicate rows with idempotent ON → skipped.
- Mixed valid/invalid rows → partial import works.

B) CSV Export

Rules

- Export uses the same template columns.
- Filters: company (if applicable), brand, date range, optional status.
- Multi-channel posts export as a single cell with | delimiter.
- Export is round-trip safe where possible.

Acceptance Criteria

- Export respects filters and permissions.
- Exported CSV can be re-imported without losing essential fields.

QA Test Cases

- Export brand A only and verify no other brands included.
- Export date range and confirm boundaries inclusive.
- Round-trip test: export then import in idempotent mode → no duplicates.

C) Filtering Requirements (end-to-end)

- Kanban, exports, and post list endpoints all accept:
    - brand_id, status_id, date range, keyword
- Filters must be scoped by organization and permissions.

G) Data Model (Supabase/Postgres)

Tenant Model Decision (MVP)

Option A: Organization == Company; Brands belong to Organization.

- Pros: simpler MVP, fewer joins, easier filtering.
- Company UI can still exist but acts as org metadata.

Tables (minimum)

organizations

Field	Type	Notes
id	uuid PK	
name	text	
default_timezone	text	e.g., Asia/Manila
default_post_time	time	e.g., 09:00
created_at	timestamptz	

Index: id

brands

Field	Type	Notes
id	uuid PK	
organization_id	uuid FK	organizations.id
name	text	unique per org (case-insensitive)
is_active	boolean	default true
created_at	timestamptz	

Indexes:

- (organization_id, lower(name)) unique

profiles (internal users)

(Use Supabase auth.users for auth; keep profile data here.)

Field	Type
id	uuid PK (matches auth.users.id)
organization_id	uuid FK
display_name	text
email	text
created_at	timestamptz

Index: (organization_id)

organization_memberships

Field	Type	Notes
id	uuid PK	
organization_id	uuid FK	
user_id	uuid FK	profiles.id
role	text	enum: owner, manager, contributor
created_at	timestamptz	

Index: (organization_id, user_id) unique

client_contacts

(Client identities for email approvals; may or may not have auth accounts.)

Field	Type	Notes
id	uuid PK	
organization_id	uuid FK	
name	text	
email	text	

created_at	timestamptz	

Index: (organization_id, lower(email)) unique

client_assignments

Defines what a client can access.

Field	Type	Notes
id	uuid PK	
organization_id	uuid FK	
client_contact_id	uuid FK	
brand_id	uuid FK	nullable if assigned per post
can_approve	boolean	default true
can_comment	boolean	default true
created_at	timestamptz	

Index: (organization_id, client_contact_id, brand_id)

post_statuses

Field	Type	Notes
id	uuid PK	
organization_id	uuid FK	
name	text	
sort_order	int	
is_active	boolean	
is_approved_terminal	boolean	if true, post considered approved state
created_at	timestamptz	

Index: (organization_id, sort_order)

posts

Field	Type	Notes
id	uuid PK	
organization_id	uuid FK	
brand_id	uuid FK	
status_id	uuid FK	
theme	text	
framework	text	optional
content_type	text	
channels	text[]	
target_audience	text	
primary_goal	text	
cta	text	
promotion_type	text	
highlight_text	text	
cross_promo_text	text	
post_date	date	nullable if TBD
post_time	time	nullable
is_tbd	boolean	default true if no date
design_deadline_at	timestamptz	nullable
content_deadline_at	timestamptz	nullable
timezone	text	default org timezone
created_by	uuid	profiles.id
created_at	timestamptz	

updated_at	timestamptz	

Indexes:

- (organization_id, brand_id)
- (organization_id, status_id)
- (organization_id, post_date)
- GIN index on channels if needed

post_assets

Field	Type	Notes
id	uuid PK	
organization_id	uuid FK	
post_id	uuid FK	
version_number	int	increments
storage_path	text	Supabase Storage object path
file_name	text	
mime_type	text	
file_size	bigint	
checksum	text	optional
is_current	boolean	
uploaded_by	uuid	profiles.id
created_at	timestamptz	

Index: (post_id, is_current)

post_caption_versions

Field	Type
id	uuid PK

organization_id	uuid FK
post_id	uuid FK
version_number	int
caption_text	text
is_current	boolean
updated_by	uuid
created_at	timestamptz

Index: (post_id, is_current)

post_comments

Field	Type	Notes
id	uuid PK	
organization_id	uuid FK	
post_id	uuid FK	
parent_id	uuid FK	nullable for top-level
author_type	text	enum: internal_user, client_contact, system
author_user_id	uuid	nullable
author_client_contact_id	uuid	nullable
body	text	
edited_at	timestamptz	nullable
deleted_at	timestamptz	nullable (soft delete)

created_at	timestamptz	

Index: (post_id, created_at)

post_activity_log

Field	Type
id	uuid PK
organization_id	uuid FK
post_id	uuid FK
event_type	text
actor_type	text
actor_user_id	uuid
actor_client_contact_id	uuid
metadata	jsonb
created_at	timestamptz

Indexes:

- (post_id, created_at)
- (organization_id, event_type)

post_approvers (internal supervisors)

Field	Type
id	uuid PK
organization_id	uuid FK
post_id	uuid FK
user_id	uuid FK
required	boolean

created_at	timestamptz

Index: (post_id, user_id) unique

post_client_approvers

Field	Type
id	uuid PK
organization_id	uuid FK
post_id	uuid FK
client_contact_id	uuid FK
required	boolean
created_at	timestamptz

Index: (post_id, client_contact_id) unique

approval_requests

Represents a “send approval email” event per recipient.

Field	Type	Notes
id	uuid PK	
organization_id	uuid FK	
post_id	uuid FK	
recipient_type	text	internal_user / client_contact
recipient_user_id	uuid	nullable
recipient_client_contact_id	uuid	nullable
requested_by	uuid	internal user
request_scope	text	supervisor / client
token_hash	text	store hash only

token_expires_at	timestamptz	
sent_at	timestamptz	
delivery_status	text	optional
created_at	timestamptz	

Index: (post_id, recipient_type, recipient_user_id, recipient_client_contact_id, sent_at)

approvals

Records decisions (from email or in-app).

Field	Type
id	uuid PK
organization_id	uuid FK
post_id	uuid FK
approval_request_id	uuid FK nullable
approver_type	text
approver_user_id	uuid nullable
approver_client_contact_id	uuid nullable
decision	text
decided_at	timestamptz
source	text
created_at	timestamptz

Index: (post_id, approver_type, approver_user_id, approver_client_contact_id)

csv_import_jobs

Field	Type

id	uuid PK
organization_id	uuid FK
uploaded_by	uuid
file_name	text
status	text
total_rows	int
created_count	int
skipped_duplicates_count	int
failed_count	int
errors_json	jsonb
created_at	timestamptz

csv_import_job_rows (optional but recommended)

Field	Type
id	uuid PK
import_job_id	uuid FK
row_number	int
raw_json	jsonb
status	text
error_message	text
created_post_id	uuid nullable

Relationships

- organizations 1→many brands, users, statuses, posts, client contacts
- brands 1→many posts
- posts 1→many assets, captions, comments, activity logs, approvals

- posts many→many approvers via post_approvers and post_client_approvers

RLS Policies Outline (high-level)

- **profiles**: user can read own; org members can read same org.
- **posts**:
  - internal users: can read/write where organization_id = membership.organization_id
  - clients: cannot read posts directly via auth unless they have client auth (phase 2). For MVP, email token endpoints serve limited views.
- **post_comments**:
  - internal: read/write same org
  - client: write via token-based endpoint that validates assignment (see API)
- **brands/statuses**: read same org; write only owner/manager
- **approval_requests/approvals**: internal reads same org; write through server only.

Storage Buckets Structure

Bucket: sokmed-post-assets
Path pattern:

- {organization_id}/{post_id}/v{version_number}/{filename}

Migration Approach

- SQL migrations via Supabase CLI.
- Seed default statuses on org creation:
  - Planning, To Do, Designing, For Review, Revision, Approved

H) API Specification (Node.js, REST)

Backend Framework Choice

Fastify (recommended for MVP):

- Better performance and built-in schema validation

- Clean plugin architecture for auth, rate limits, and validation

Auth

- Internal users authenticate via Supabase Auth; frontend sends Supabase JWT in Authorization: Bearer <token>.
- API verifies JWT using Supabase JWT secret / JWKS.

Common Response Format

```
{
  "data": {},
  "error": null,
  "request_id": "uuid"
}
```

Rate Limiting

- Global: 100 req/min per user
- Sensitive endpoints:
    - approval token endpoints: 20 req/min per IP
    - import preview/confirm: 10 req/min per user

Endpoint Groups

1) Companies/Brands (CRUD, admin)

GET /v1/brands

- Query: ?is_active=true
- Response 200:

```
{"data":[{"id":"...","name":"Brand A"}],"error":null}
```

POST /v1/brands (Owner/Manager)

```
{"name":"Brand A"}
```

201 created

PATCH /v1/brands/:id

{"name":"New Name","is_active":true}

2) Statuses (Admin)

GET /v1/statuses
POST /v1/statuses

{"name":"For Review","sort_order":4}

PATCH /v1/statuses/:id
DELETE /v1/statuses/:id

- If posts exist: 409 with message requiring remap.

3) Posts CRUD

GET /v1/posts
Query params:

- brand_id, status_id, date_from, date_to, q
  Response: list for Kanban

POST /v1/posts

```
{
  "brand_id":"uuid",
  "theme":"March Promo",
  "content_type":"Banner",
  "channels":["FB","IG"],
  "target_audience":"Freelancers",
  "primary_goal":"Drive inquiries",
  "cta":"Message us",
  "promotion_type":"Discount",
  "post_date":"2026-03-15",
  "post_time":"09:00",
  "is_tbd":false,
  "design_deadline_at":"2026-03-10T01:00:00Z",
  "content_deadline_at":"2026-03-09T01:00:00Z"
}
```

201 created

GET /v1/posts/:id
Returns post + current asset + current caption + approvers + client approvers

PATCH /v1/posts/:id
Metadata updates with field-level validation

4) Move status (kanban)

POST /v1/posts/:id/move-status

{"to_status_id":"uuid"}

- 200 returns updated post
- Writes audit event

5) Upload design (signed URL flow) + asset versioning

POST /v1/posts/:id/assets/upload-url
Request:

{"file_name":"banner.png","mime_type":"image/png","file_size":345678}

Response 200:

{
  "data":{
    "upload_url":"https://signed-url",
    "storage_path":"org/post/v3/banner.png",
    "asset_version_number":3
  },
  "error":null
}

Frontend uploads directly to Supabase Storage using signed URL.

POST /v1/posts/:id/assets/confirm

{
  "storage_path":"org/post/v3/banner.png",

```
  "file_name":"banner.png",
  "mime_type":"image/png",
  "file_size":345678,
  "checksum":"optional"
}
```

Creates post_assets record, sets previous current to false, sets new to current, writes audit.

6) Caption versioning

POST /v1/posts/:id/captions

```
{"caption_text":"New caption..."}
```

Creates new version, marks current, writes audit.

GET /v1/posts/:id/captions
Returns versions list.

7) Comments (threaded)

GET /v1/posts/:id/comments
POST /v1/posts/:id/comments

```
{"parent_id":null,"body":"Please adjust the CTA."}
```

PATCH /v1/comments/:id (edit)

```
{"body":"Updated text","edit_reason":"typo"}
```

DELETE /v1/comments/:id (soft delete)

8) Request approval (supervisor/client)

POST /v1/posts/:id/request-approval

```
{
  "scope":"supervisor",
  "recipient_user_ids":["uuid1","uuid2"],
  "mode":"all_required"
```

```
}
```

or

```
{
  "scope":"client",
  "recipient_client_contact_ids":["uuid1"],
  "mode":"all_required"
}
```

Response: 200 with request IDs.

Rules:

- Creates approval_requests per recipient
- Generates **single-use token**, stores hash in DB, sends email
- Writes audit event post.approval_requested

9) Approve via email link (token-based)

POST /v1/email-approve

```
{"token":"raw-token-from-email","decision":"approved"}
```

Response 200:

```
{"data":{"post_id":"uuid","decision":"approved","decided_at":"..."},"error":null}
```

Idempotency:

- token can be used once; subsequent use returns 409 “Token already used”
- also prevent double approvals by same approver: return 200 with existing decision

10) Audit log retrieval

GET /v1/posts/:id/activity
Returns ordered audit events.

11) CSV import start/preview/confirm + export endpoint

POST /v1/imports/csv/preview

- multipart upload OR signed upload then reference
  Response:


```
{
  "data":{
    "import_job_id":"uuid",
    "preview_rows":[{"row":1,"valid":true,"mapped":{...}}],
    "errors":[{"row":2,"message":"Missing Date"}]
  },
  "error":null
}
```


POST /v1/imports/csv/confirm

{"import_job_id":"uuid","idempotent":true}

GET /v1/exports/csv
Query:

- brand_id, date_from, date_to, status_id
  Returns CSV file download.

I) Email Templates + Approval Link Flow

Provider (MVP choice)

Resend (simple API, good dev experience). (See section 6 for details.)

Email Content Structure (exact)

Subject

- [SokMed] Approval needed: <Brand> – <Theme> (<Post Date or TBD>)

Body

1. Header:
    - Brand, Theme
    - Post Date/Time (or TBD)
2. Design preview:
    - Embedded image (or hosted image with secure link)
3. Caption:
    - Plain text block
4. Actions (buttons):
    - **Approve**
    - **Comment**
5. Footer:
    - "You received this because you're an approver for ."
    - Link to open post in SokMed

Deep Link Strategy

Approve (no login required)

- Email includes:
    - https://app.sokmed.com/email/approve?token=<rawToken>
- Frontend approve page calls API POST /v1/email-approve with token and decision.

Comment (login optional)

- Email includes:
    - https://app.sokmed.com/p/<postId>?focus=comment&token=<optionalToken>
      Rules:
- If recipient is a client without auth:
    - MVP allows **token-based comment posting** (recommended for frictionless UX):
        - Comment page loads a lightweight "Client Comment View" when token present.
        - Posting comment calls POST /v1/email-comment with token + body.
- If recipient is internal user:

    - Must be logged in; if not, login then redirect to post with focus=comment.

Token Security

- Token is random 32+ bytes, base64url.
- Store only token_hash = sha256(token + server_pepper) in DB.
- Token expiration:
    - Default: 7 days
    - Configurable per org later
- Replay protection:
    - approval_requests has used_at set on success (or approvals table references token).
    - If used again → 409.

Handling Forwarded Emails

Behavior (MVP): Forwarded email **still works** while token valid.
Mitigation:

- Token is single-use; once used, forward becomes useless.
- If risk is unacceptable, add "verify recipient email" step later (phase 2).

Multiple Approvers & Overall Approval State

- Approval mode: **ALL required**
- Post is considered fully approved when:
    - all required internal approvers have approved
    - and all required client approvers have approved (if client approval requested)
- When fully approved:
    - system can auto-move status to "Approved" (configurable; MVP default ON)

Sample Email (text)

Approval needed: Brand A – March Promo (2026-03-15 09:00 Asia/Manila)

[Design Preview Image]

Caption:
"March is here! Get 20% off... CTA: Message us to book."

[Approve] [Comment]

Open in SokMed: <link>

J) QA Test Plan

Test Matrix by Feature

Feature	Areas	Priority
Kanban	load, filters, drag/drop, concurrency	P0
Post Detail	view, upload, caption versioning, comments	P0
Approvals	email send, approve token, comment link, idempotency	P0
Audit	completeness, ordering, actor correctness	P0
CSV Import	parsing, validation, preview, confirm, duplicates	P0
CSV Export	filters, formatting, round-trip	P1
Admin	status editor, brand mgmt, permissions	P1
Security	RLS, cross-tenant isolation, token abuse	P0

Happy Paths + Negative Tests (samples)

Kanban

- HP: drag card Planning → Designing → For Review; verify audit events each time.
- NEG: contributor tries to delete status; should fail.
- NEG: client attempts to access /kanban; should be denied.

Post Detail

- HP: upload banner, edit caption, comment, request approval.
- NEG: upload .exe file; reject.
- NEG: set deadline in the past (non-owner); reject.

Email approve/comment

- HP: click approve link → decision recorded, token invalidated, audit logged.
- NEG: token expired → 401 with message “Link expired”.
- NEG: token reused → 409 “Already used”.
- NEG: token valid but post not assigned to that client (if token tied to request) → 403.

Audit verification

- For each action, confirm one audit entry with correct metadata.

CSV import/export

- HP: import valid CSV → posts created in Planning.
- NEG: missing Brand → row invalid.
- NEG: unknown brand with no create permission → map required; cannot proceed.
- Round-trip: export then import idempotent → 0 new posts.

Permission Tests (per role)

- Owner can manage statuses/brands/users.
- Manager can request approvals and edit posts.
- Contributor can edit design/caption and request approvals.
- Client can only approve/comment on assigned posts.

Cross-tenant isolation tests

- Create Org A and Org B; ensure Org A user cannot read any Org B posts by direct API calls or by guessed IDs.

K) DevOps / Deployment Notes

Environments

- **dev**: local + Supabase dev project
- **staging**: separate Supabase project + staging app domains
- **prod**: separate Supabase project + production domains

Deployment Approach (DevOps-friendly)

- Frontend (React/TS): **Vercel**
- Backend (Fastify API): **Render** or **Fly.io**
- Supabase: managed Postgres/Auth/Storage

Environment Variables (minimum)

Frontend:

- VITE_SUPABASE_URL
- VITE_SUPABASE_ANON_KEY
- VITE_API_BASE_URL

Backend:

- SUPABASE_URL
- SUPABASE_SERVICE_ROLE_KEY (server-only)
- SUPABASE_JWT_SECRET or JWKS config
- RESEND_API_KEY
- APP_BASE_URL (for links)
- TOKEN_PEPPER (secret for hashing)
- RATE_LIMIT_* config

Secrets Handling

- Use platform secret stores (Vercel/Render).
- Never expose service role keys to frontend.
- Rotate keys periodically.

CI/CD Basics

- GitHub Actions:
    - lint + typecheck + tests on PR

    - deploy staging on merge to main
    - deploy prod on tagged release

Logging/Monitoring

- Structured JSON logs (pino)
- Capture request_id, org_id, user_id (if present)
- Error tracking (Sentry recommended)

Backups/Migrations/Rollback

- Supabase scheduled backups (enable)
- Use migration scripts; rollback by down migrations where possible
- For storage: lifecycle rules for old asset versions (optional)

L) Security + Compliance Checklist

Security

- ✅ Supabase RLS enabled on all tables
- ✅ Principle of least privilege:
    - client actions only via token endpoints
    - internal actions require JWT
- ✅ Secure token handling:
    - store hash only, expiration, single-use
- ✅ XSS protection:
    - sanitize/escape comment rendering
    - enforce Content Security Policy (CSP) on frontend
- ✅ File upload validation:
    - allowlist mime types: image/png, image/jpeg, image/webp
    - max size (e.g., 10MB)
    - virus scanning optional (phase 2)
- ✅ Rate limiting on approval endpoints
- ✅ Audit integrity:
    - append-only log; restrict delete/update via RLS + server
    - optional: database trigger preventing updates to audit rows (recommended)

Compliance / Data Retention

- Define retention:
  - Keep audit logs for 2 years (configurable)
  - Keep old asset versions for 6 months (optional lifecycle)
- GDPR-like:
  - ability to delete client contact data while preserving audit actor as anonymized ID (phase 2)

M) Implementation Plan (Phased)

Phase 0: Foundation (Dependencies)

1. Supabase project setup (dev/staging/prod)
2. DB schema + migrations + RLS scaffolding
3. Auth flow (internal users) + organization membership bootstrap
4. Base API skeleton (Fastify + Supabase client)

Phase 1: Core MVP (Implementation-ready)

Backend

- Posts CRUD + filtering
- Statuses CRUD + reorder
- Assets signed upload + confirm + versioning
- Caption versioning endpoints
- Comments endpoints (threaded)
- Activity log writing (central middleware/service)
- Approval request service:
  - create approval_requests
  - generate tokens + hash store
  - send emails
- Email approval token service:
  - verify token (hash match, not expired, not used)
  - idempotency logic
  - write approvals + audit
  - compute overall approval state, auto-move to Approved

Frontend

- Login + session handling
- Kanban board UI with filters + drag/drop
- Post detail page with:
  - preview, caption edit, version history
  - comments
  - activity tab
  - notify buttons
- Admin screens:
  - status editor
  - brand management
  - user/client management (basic)

CSV Pipeline

- CSV preview parsing endpoint + validation rules
- confirm import endpoint + idempotency
- job tracking tables + UI preview table
- export endpoint + UI modal

Phase 2: Enhancements

- In-app notifications
- Internal-only comments vs client-visible comments
- Client viewer role
- Webhooks for email delivery/bounces
- More granular contributor scope (brand/team-based)
- Diff view for caption changes

6) Email Sending Requirement (Provider + Implementation)

Provider Choice: Resend

How Node triggers emails

- Backend uses Resend API client:
  - resend.emails.send({ from, to, subject, html, text })
- Emails sent when POST /v1/posts/:id/request-approval is called.

Template Management

- MVP: store templates as code (React email or handlebars).
- Recommended: **react-email** (component-based templates) compiled to HTML.

Env Vars

- RESEND_API_KEY
- EMAIL_FROM (e.g., SokMed <approvals@yourdomain.com>)
- APP_BASE_URL

Deliverability Notes

- Set SPF/DKIM for sending domain.
- Use a dedicated subdomain (e.g., mail.yourdomain.com) optionally.
- Keep consistent From name/address.

Bounces Handling (Optional MVP)

- Add webhook endpoint POST /v1/webhooks/resend:
  - update approval_requests.delivery_status
  - log audit event email.bounced

7) Assumptions & Open Questions

Assumptions Made

1. **Organization == Company (Option A)** for MVP; brands belong directly to organization.
2. **Approval mode = ALL required** for MVP; configurable later.
3. Clients do not have full authenticated accounts in MVP; they interact via **email tokens**.
4. Only **approval request emails** are required in MVP; comment notifications are phase 2.
5. Status transitions are unrestricted (any → any) in MVP; stricter rules later.

6. Contributors can see all posts in the organization in MVP; finer access controls later.

Open Questions (would normally ask, but build can proceed)

1. Should client comments be visible to all internal users by default (currently yes)?
2. Do we need “Request Changes” as a first-class decision button in email (recommended)?
3. Do you want the post to auto-move to “Approved” only after both supervisor + client approvals, or separate terminals?
4. Should “Notify Supervisor” be allowed if no design/caption exists, or block until minimum completeness?
5. Any legal/compliance constraints for storing client feedback and email addresses (retention policy)?

Concrete Examples (requested)

Sample API: Request Supervisor Approval

POST /v1/posts/123/request-approval

```
{
  "scope":"supervisor",
  "recipient_user_ids":["u1","u2"],
  "mode":"all_required"
}
```

200

```
{
  "data":{
    "post_id":"123",
    "requests":[
      {"id":"r1","recipient":"u1","sent_at":"2026-03-03T02:10:00Z"},
      {"id":"r2","recipient":"u2","sent_at":"2026-03-03T02:10:01Z"}
    ]
  },
```

"error":null,
"request_id":"req-abc"
}

Sample API: Email Approve

POST /v1/email-approve

{"token":"XyZ...","decision":"approved"}

Sample CSV Row (template)

Date,Brand,Highlight (80%),Cross-Promo (20%),Theme,Content Type,Channels,Target Audience,Primary Goal,CTA,Type of Promotion
2026-03-15,Brand A,Promo focus,Partner mention,March Promo,Banner,FB|IG,Freelancers,Drive inquiries,Message us,Discount